

Faire tourner EnTraineR : un exemple reproductible

Interruption de jeu

Pourquoi cette interruption ?

Cette fiche montre comment faire tourner EnTraineR sur un exemple complet, reproductible et indépendant du plateau.

L'objectif n'est pas encore d'appeler un modèle de langage. L'objectif est de vérifier que l'on sait produire un prompt inspectable.

```
données simulées  
→ analyse statistique  
→ objet R  
→ fonction EnTraineR  
→ prompt contrôlé  
→ génération optionnelle
```

Le principe à retenir

EnTraineR ne remplace pas l'analyse statistique.

Il intervient **après** la production d'un résultat statistique, pour organiser une demande d'interprétation adressée à un modèle de langage.

Le point central est donc :

```
Je ne donne pas des données brutes au modèle.  
Je donne un résultat statistique explicite et un prompt contrôlé.
```

L'option clé est :

```
generate = FALSE
```

Elle permet de construire le prompt sans appeler de modèle de langage.

Installer ou charger les packages

Cette fiche utilise FactoMineR pour produire les sorties statistiques et EnTraineR pour construire les prompts.

```
# À faire une seule fois si nécessaire
install.packages("FactoMineR")

# Si EnTraineR est installé depuis GitHub
install.packages("remotes")
remotes::install_github("Sebastien-Le/EnTraineR")
```

Puis on charge les packages.

```
library(FactoMineR)
library(EnTraineR)
```

Vérifier les fonctions disponibles

Selon la version installée, les noms exacts des fonctions peuvent varier légèrement. On peut inspecter les fonctions exportées.

```
fonctions_entrainer <- grep(
  "^trainer_",
  getNamespaceExports("EnTraineR"),
  value = TRUE
)

fonctions_entrainer
```

Cette étape est importante : elle distingue le support de cours de l'environnement R réellement disponible sur la machine.

Créer un petit jeu de données

On crée un jeu de données simulé.

Chaque ligne représente un répondant. On dispose de variables quantitatives, comme `satisfaction`, `prix_percu` ou `intention_achat`, et de variables qualitatives, comme `type_produit` ou `budget_contraint`.

```
set.seed(123)

n <- 120

questionnaire_demo <- data.frame(
  satisfaction = round(rnorm(n, mean = 6.2, sd = 1.2), 1),
  prix_percu = round(rnorm(n, mean = 5.5, sd = 1.4), 1),
```

```

naturalite = round(rnorm(n, mean = 6.0, sd = 1.1), 1),
type_produit = factor(
  sample(
    c("Classique", "Végétal", "Local"),
    n,
    replace = TRUE
  )
),
budget_constraint = factor(
  sample(
    c("Faible", "Moyen", "Fort"),
    n,
    replace = TRUE
  )
)
)

questionnaire_demo$intention_achat <- round(
  1.5 +
  0.65 * questionnaire_demo$satisfaction -
  0.25 * questionnaire_demo$prix_percu +
  0.20 * questionnaire_demo$naturalite +
  rnorm(n, mean = 0, sd = 0.9),
  1
)

head(questionnaire_demo)

```

Ce jeu de données n'a pas vocation à être réaliste. Il sert à produire des sorties statistiques manipulables.

Exemple 1 : une régression avec LinearModel()

On commence par un modèle linéaire simple.

La variable à expliquer est `intention_achat`. Les variables explicatives sont `satisfaction`, `prix_percu` et `naturalite`.

```

formule_lm <- intention_achat ~ satisfaction + prix_percu + naturalite

res_lm <- FactoMineR::LinearModel(
  formule_lm,
  data = questionnaire_demo,
  selection = "none"
)

names(res_lm)

```

L'objet `res_lm` contient plusieurs composantes. Les plus utiles pour l'interprétation sont souvent :

```
res_lm$Ftest  
res_lm$Ttest
```

Ces sorties sont encore des sorties statistiques. EnTraineR intervient ensuite pour construire un prompt à partir de cet objet.

Produire un prompt avec EnTraineR

L'appel le plus important est le suivant.

```
prompt_lm <- EnTraineR::trainer_linear_model(  
  res_lm,  
  generate = FALSE  
)  
  
prompt_lm
```

Le rôle de `generate = FALSE` est central : on demande à EnTraineR de construire le prompt, mais pas d'appeler un modèle de langage.

Exemple 2 : une ANOVA avec AovSum()

On produit maintenant une ANOVA avec un effet principal et une interaction.

```
formule_aov <- satisfaction ~ type_produit * budget_constraint  
  
res_aov <- FactoMineR::AovSum(  
  formule_aov,  
  data = questionnaire_demo  
)  
  
names(res_aov)
```

On peut inspecter les deux niveaux de sortie.

```
res_aov$Ftest  
res_aov$Ttest
```

Le `Ftest` aide à lire les effets globaux. Le `Ttest` aide à lire les coefficients ou contrastes.

Produire un prompt pour l'ANOVA

Selon la version d'EnTraineR, la fonction peut s'appeler `trainer_AovSum()` ou `trainer_aovsum()`.

```
prompt_aov <- trainer_aovsum(
  res_aov,
  generate = FALSE
)

prompt_aov
```

Là encore, l'important est de produire un prompt avant toute génération.

Inspecter les options disponibles

Les options exactes dépendent de la fonction et de la version du package.

On peut les inspecter avec `formals()`.

```
formals(trainer_linear_model)
```

Cette commande permet de vérifier si la fonction accepte des arguments comme :

- `generate` ;
- `model` ;
- `engine` ou `llm_engine` ;
- `audience` ;
- `language` ;
- `summary_only`.

Il ne faut pas supposer qu'une option existe : il faut vérifier les arguments disponibles.

Appeler un modèle de langage : une étape optionnelle

Dans le cadre du plateau, on travaille surtout avec `generate = FALSE`.

L'appel direct à un modèle de langage doit rester optionnel.

```
# Exemple indicatif seulement.
# À utiliser seulement si le moteur LLM est configuré sur votre machine.

prompt_lm_genere <- trainer_linear_model(
  res_lm,
  generate = TRUE,
  llm_model = "llama3",
  llm_engine = "ollama"
)
```

Avant d'utiliser `generate = TRUE`, il faut vérifier :

- que le moteur demandé est disponible ;
- que le modèle est installé ou accessible ;
- que le prompt obtenu avec `generate = FALSE` est correct ;
- que les consignes ne demandent pas au modèle d'inventer des résultats.

Comparer méthode manuelle et méthode EnTraineR

Dans le plateau, les cases précédentes ont montré comment construire un prompt avec `capture.output()` et `paste()`.

La logique manuelle est instructive, car elle rend visibles les étapes :

```
sortie statistique
→ capture de la sortie
→ construction du prompt
→ inspection du texte
```

EnTraineR automatise cette logique.

```
comparaison <- data.frame(
  methode = c(
    "Prompt manuel",
    "Prompt EnTraineR"
  ),
  geste_R = c(
    "capture.output() + paste()",
    "trainer_*(..., generate = FALSE)"
  ),
  avantage = c(
    "comprendre toutes les étapes",
    "industrialiser la construction du prompt"
  ),
  stringsAsFactors = FALSE
)

comparaison
```

Le prompt manuel sert à comprendre. EnTraineR sert à stabiliser et accélérer.

Bonnes pratiques

Avant d'utiliser une sortie produite par EnTraineR, il faut vérifier le prompt.

Questions à se poser :

- Le contexte statistique est-il clair ?
- La variable à expliquer est-elle identifiée ?
- Les résultats transmis sont-ils bien ceux que l'on souhaite interpréter ?
- Les limites d'interprétation sont-elles explicites ?
- Le prompt interdit-il au modèle d'inventer des résultats ?
- Le niveau de langage correspond-il au public visé ?

Le bon usage d'EnTraineR n'est donc pas :

```
faire générer plus vite
```

mais plutôt :

```
faire circuler des résultats statistiques dans un cadre contrôlé
```

À retenir

EnTraineR est utile parce qu'il rend reproductible une étape souvent artisanale : transformer une sortie statistique en prompt.

Le workflow recommandé est :

```
produire l'analyse  
→ inspecter la sortie  
→ construire le prompt avec generate = FALSE  
→ vérifier le prompt  
→ générer seulement si nécessaire
```